BondNest

System Documentation

A social networking web application with posts, moderation, messaging, and notifications

Technical record of the implemented system
September 2026

Author
Tolentino, Lawrence Dave P.

Live application: https://bondnest.up.railway.app/
GitHub: https://github.com/rproject1324/BondNest

Table of Contents

1. Project Overview

BondNest is an account-based social networking web application built with vanilla PHP and PostgreSQL. Registered users create text/image posts, like and comment, exchange direct messages, and receive moderation notifications. Administrators review content through a dashboard that supports approving, placing on hold, warning, and deleting posts.

Important context. BondNest was developed as a **second-year level, unfinished project**. It is not feature-complete: several flows are partial, debug/test artifacts remain in the repository, and some pages contain dead links or stub scripts. This document records the system **as implemented**, including its gaps, so future maintainers and reviewers see an accurate picture rather than an idealized one.

Item	Current value
Project name	BondNest
Project type	Full-stack web application (server-rendered PHP, no framework)
Intended users	Students / general users (campus social network)
Primary live host	Railway (PHP built-in server + PostgreSQL)
Current status	Deployed but unfinished; known limitations documented later

2. Objectives

2.1 General objective

To provide a small social platform where users can publish posts, interact through likes, comments and messages, and where administrators can moderate content through approval, hold, warning, and deletion workflows with user-facing notifications.

2.2 Specific objectives

- Support registration with email OTP verification, session login, and password recovery.
- Provide post creation with images, likes, threaded comments/replies, and personal profiles.
- Provide 1:1 direct messaging with unread tracking and user search.
- Give administrators a dashboard with statistics, post search, and moderation actions.
- Notify users of moderation outcomes (approved, on hold, warned, deleted) with detail views.
- Enforce a shared password-strength policy on signup, reset, and password change.
- Deploy the stack on Railway with PostgreSQL and persistent uploads.

3. System Features and Functionalities

3.1 Guests (unauthenticated)

- Register with username, email, names, gender, birthday, and a policy-checked password; must accept Terms & Privacy.
- Verify the account with a six-digit email OTP (Brevo) on the verification page.
- Sign in with username or email plus password; recover a forgotten password via email OTP.

3.2 Authenticated users

- Homepage feed of approved/posted posts with create/edit/delete of own posts and image upload.
- Likes with toggle state and counts; comments with replies, edit, and delete.
- Profile pages (own + others) with photo, bio, details, and per-user post lists.
- Settings: personal info, profile photo, bio, email change via OTP, password change.
- Direct messaging inbox, conversation view, user search, read markers, unread badges.
- Notifications bell (5 per page, paginated): post approved / on hold / warning / deleted, each linking to a detail inbox with Clear-Read support.
- Online presence (Active now / Away / Last seen) derived from activity heartbeats.

3.3 Administrator

A user whose email matches the configured admin email (or whose *is_admin* flag is set) opens *admin.php*: statistics cards, post table with search/filter, and approve / hold / delete (with reason) / warn (with reason) actions. Deletions and warnings archive a snapshot of the post for later detail views. Admin is a configured operator role, not a public self-serve role.

3.4 Cross-cutting behavior

- PHP sessions gate every authenticated page; guests are redirected to the login page.
- Post age labels ("N seconds ago") are server-rendered, then kept live by a server-clock-synced client timer.
- Uploads persist under UPLOADS_DIR (intended for a Railway volume).
- Toast notifications confirm actions across homepage, profile, and admin flows.

4. Technology Stack

Category	Technology / service
Frontend	Server-rendered PHP templates, custom CSS, vanilla JavaScript; Bootstrap Icons 1.11, Font Awesome 6
Backend	Vanilla PHP 8.1+ (deployed on PHP 8.3) + PDO; no framework; idempotent migrate.php bootstrap
Database	PostgreSQL on Railway (psycopg2-style via pdo_pgsql); MySQL locally under XAMPP
Authentication	PHP sessions, password_hash()/password_verify(), six-digit email OTPs
Email / OTP	Brevo transactional HTTP API (api.brevo.com/v3/smtp/email)
Version control	Git / GitHub
Hosting	Railway (web + Postgres + volume); PHP built-in server via Procfile

5. System Design and Architecture

5.1 Runtime architecture

The browser interacts with PHP pages that authenticate the session, read/write PostgreSQL through PDO, store images on a volume-backed uploads directory, and send mail through Brevo. Admin moderation writes both the post row and a notification row (plus snapshot rows for warnings/deletions). A small set of JSON endpoints serves likes, comments, messages, notification polling, and admin AJAX.

Browser (PHP pages + JS polling) → PHP on Railway (sessions, PDO) → PostgreSQL; PHP → Brevo for OTP mail; uploads → Railway volume.

5.2 Why this shape

The project prioritizes a small, readable codebase over framework ceremony: one PHP file per page, one endpoint per action, and a single migration script that creates/patches tables on boot. The trade-off is duplication (notably the profile page variants) and the rough edges listed in chapter 15.

6. Major Workflows

6.1 Registration and email OTP

Signup validates fields and the password policy, stores a hashed password in *pending_registrations* with a six-digit OTP expiring in 10 minutes, and mails the code via Brevo (dev-mode logs it when keys are absent). Verification creates the *users* row, starts a session, and routes to the homepage (or admin).

6.2 Login and password recovery

Login accepts username or email (case-insensitive) plus password with a generic failure message. Forgotten passwords use a three-step Brevo OTP flow (request → verify → reset under the password policy).

6.3 Posting and moderation

Users publish text/image posts (status *posted*). Admins approve (→ *approved*), hold (→ *on-hold*), warn (snapshot into *warnings* + notification), or delete (snapshot into *deleted_posts* + notification, then row removal). Approve/hold notifications additionally embed a post snapshot so detail views render even if the post row later disappears. Live status polling updates badges without refresh.

6.4 Messaging and notifications

Direct messages store sender/receiver/content with read flags and soft delete. Moderation outcomes create typed notifications; the navbar bell fetches 5 per page, marks displayed items read, and deep-links each type to its inbox (warnings / held / approved / deleted).

7. Database Design

Production uses PostgreSQL via DATABASE_URL; local development defaults to MySQL on XAMPP. *migrate.php:runMigration()* creates tables if missing and patches columns on boot. Time is stored in UTC.

Table	Role
users	Identity, password hash, profile fields, last_activity/user_status, is_admin
posts	Content, image_path, likes counter, created/updated_at, moderation status
comments	Post comments with parent_id for replies
likes	Dedup join UNIQUE(user_id, post_id)
notifications	Typed inbox (post_warning/deleted/on_hold/approved), message, reference_id, is_read, additional_data
messages	1:1 messages with image_path, read flags, soft delete
warnings	Snapshot archive of warned posts + warning_reason
deleted_posts	Snapshot archive of deleted posts + deletion_reason
admin_actions	Audit log of admin actions with comments
pending_registrations	Unverified signups keyed by email with OTP + expiry
password_resets	Forgot-password OTPs keyed by email
email_change_challenges	Logged-in email-change OTP challenges

8. User Interface and Page Documentation

Sign In (index.php)

Access. Public.

Main UI. Login card plus in-page forgot-password OTP, new-password, and status messaging.

Actions. Submit credentials; request/verify/reset password via OTP.

Server interaction. Login + forgot_* actions in index.php.

Sign Up (signup.php)

Access. Public.

Main UI. Two-column identity form, live strength meter, shared policy message, Terms & Privacy modal.

Actions. Inline validation; agree-and-continue validates the whole form; success navigates to verification.

Server interaction. Signup + username-availability check.

Verify Registration (verify-registration.php)

Access. Public (pending email).

Main UI. Six-digit OTP inputs with resend.

Actions. Verify code; resend code; then signed in.

Server interaction. Verification endpoints.

Homepage (homepage.php)

Access. Signed in.

Main UI. Create-post box, moderated feed, like/comment controls, toasts.

Actions. Publish/edit/delete own posts; like; comment/reply.

Server interaction. create/update/delete_post, like_post, *_comment, get_posts_status.

Profile (profile-page.php)

Access. Signed in.

Main UI. Cover card, bio/details, own-post activity list with moderation badges.

Actions. View/edit profile and details; manage own posts.

Server interaction. Profile update + post endpoints.

Settings (settings.php)

Access. Signed in.

Main UI. Personal info, photo, bio, email-change OTP, password change, presence.

Actions. Update identity and credentials with inline validation.

Server interaction. Settings + email-challenge endpoints.

Messages (message.php)

Access. Signed in.

Main UI. Inbox, conversation thread, user search.

Actions. Send/read messages; search users.

Server interaction. get/update_message, get_messages, search_users.

Notification inboxes (warnings / held / approved / deleted_posts.php)

Access. Signed in (own only).

Main UI. Typed cards with View Details modals showing author, snapshot, image, reason, timestamps.

Actions. Open details; clear read notifications.

Server interaction. Per-type queries + mark-read endpoints.

Admin (admin.php)

Access. Admin only.

Main UI. Stat cards, searchable post table, approve/hold/delete/warn dialogs.

Actions. Moderate content with reasons; review metrics.

Server interaction. admin_actions/admin_functions + get_stats.

9. Authentication, Authorization, and Account Security

9.1 Sessions and authorization

Successful login/verification stores *user_id* (plus username/admin flag) in the PHP session. Authenticated pages redirect guests to the login page. Admin pages additionally require the admin flag or a configured admin email, else redirect to the homepage. Notification inboxes always scope rows to the viewer.

9.2 Passwords

Hashed with *password_hash()* (PASSWORD_DEFAULT) and checked with *password_verify()*. New passwords must be 12–64 characters with upper, lower, digit, and special characters, no spaces, not on a small common-password list, and must not contain the username, email, or names. The same rules run in PHP and in client-side JavaScript. Resets must additionally differ from the old password.

9.3 Email OTP

Six-digit Brevo codes cover registration, password reset, and email change, each with a 10-minute expiry. Without Brevo keys the server logs the code (dev mode) and continues so local testing is unblocked.

9.4 Admin auto-promotion

An account whose email matches BONDNEST_ADMIN_EMAIL (also DIARI_ADMIN_EMAIL / ADMIN_EMAIL, comma-separated, case-insensitive) is promoted to *is_admin = 1* during migration, login, verification, settings, and navbar checks, and routed to the admin dashboard.

10. External Services: Roles and Setup

10.1 Railway

Hosts the PHP web service and PostgreSQL, injects environment variables and PORT, and mounts a volume for uploads. Deploy connects the GitHub repo; the Procfile starts the PHP built-in server against the repo root.

10.2 Brevo

Transactional email for verification, reset, and email-change OTPs via POST api.brevo.com/v3/smtp/email with an api-key header, using the dashboard API key and verified sender.

11. Environment Variables and Configuration

Secrets live in Railway Variables, never in git.

Variable	Purpose	How configured
DATABASE_URL	PostgreSQL connection (triggers pgsql mode)	Added automatically when Postgres is linked
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASS	Postgres via parts, or local MySQL name	Operator / defaults (root, empty password locally)
UPLOADS_DIR	Persistent image directory	Railway volume path
BONDNEST_ADMIN_EMAIL (also DIARI_ADMIN_EMAIL, ADMIN_EMAIL)	Admin allow-list for auto-promotion	Railway Variables
BREVO_API_KEY	Brevo HTTP API key	Brevo dashboard
BREVO_SENDER_EMAIL	From address	Brevo verified sender

Variable	Purpose	How configured
BREVO_SENDER_NAME	From display name (default BondNest)	Operator
PORT	HTTP listen port	Provided by Railway

Do not commit real values. The screenshot-grade values seen during development (API keys, database URLs) belong in Railway Variables only.

12. Local Development Setup

- Prerequisites: PHP 8.1+ with pdo_pgsql/pdo_mysql/mbstring/fileinfo/json, plus MySQL (XAMPP) for local runs.
- Clone the repository and point the web root at the project (or run *php -S 127.0.0.1:8000*).
- Without DATABASE_URL the app uses local MySQL (*bondnest_db*, root, empty password); *migrate.php* creates/patches tables on boot.
- Leave Brevo keys unset for dev-mode OTP logging, then register and publish a post.
- Note: signup uses a Postgres-only upsert and some LIMIT/OFFSET bindings assume pgsql — Railway remains the reference runtime.

13. Railway Deployment

The repo builds via *nixpacks.toml* (php83 + pdo_pgsql + pdo_mysql) and starts with *Procfile: web: php -S 0.0.0.0:$PORT -t .* Attach Postgres for DATABASE_URL, set the Brevo/admin/upload variables, and mount a volume at UPLOADS_DIR so photos survive redeploys.

14. Testing and Current Behavior

Reviewed from the implementation plus operator-described behavior; not an automated suite.

Feature	Expected	Actual / status
Registration + OTP	Pending row + mailed code	Operational (dev-mode log without keys)
Email verification	OTP creates user row	Operational; 10-minute expiry, resend applies
Login	Session on valid password	Operational; generic failure message
Posts / likes / comments	CRUD + toggles + replies	Operational
Messaging	Send/read with badges	Operational
Admin moderation	Approve/hold/warn/delete + notify	Operational; snapshots embedded in new notifications
Notifications	Bell + paginated dropdown + inboxes	Operational
Password policy	Consistent rules everywhere	Operational (PHP + JS)
Uploads	Persist across deploys	Operational when volume mounted

15. Known Limitations and Current System Status

Area	Status
Project completeness	Unfinished second-year project; several flows partial, features missing vs. plan
Dead redirects	logout / profile / message / search_users point to process_login.php, which does not exist (login is index.php)
Debug/test leftovers	test_*.php, debug_*.php, *.log/*.txt, a create_test hook, and a visible Test link ship in the repo
Duplicate files	profile-page backup/modified/bak variants, homepage2.css, *-fix.css, web-images/homepage.php
MySQL parity	Signup upsert is Postgres-only; some bindings assume pgsql — Railway is the reference runtime
Migrated admin tool	The Migrate Deleted Posts admin utility was removed pending a rework
Old notifications	Approve/hold items created before snapshots show identity fallback; images only when the post row still exists
Security posture	Student-grade: no CSRF tokens, no rate limiting, no lockouts; generic login errors only

16. Lessons Learned

- One file per page keeps a student project readable, but shared UI (modals, validation) should be extracted before it is copied a third time.
- One-shot admin tools need the same data guarantees as live features — notifications should carry snapshots from day one.
- Environment-driven configuration (DATABASE_URL, admin email, mail keys) keeps one codebase deployable in two places.
- Client clocks cannot be trusted for timestamps; syncing to server time avoids visible jumps.
- Inline onclick handlers cannot reach functions hidden inside DOMContentLoaded closures — expose or delegate.
- Unbreakable strings need CSS-level handling (overflow-wrap + clamping), not just server truncation.

17. Future Improvements

- Fix dead redirects (process_login.php → index.php) and remove debug/test artifacts and duplicate files.
- Restore a reworked deleted-posts migration/backfill admin tool.
- Add CSRF protection, rate limiting, and login lockouts.
- MySQL parity (portable upserts/bindings) or officially drop local-MySQL support.
- Friend/follow graph, richer feed ranking, and real-time updates beyond polling.
- Automated tests and accessibility pass.

18. Project Links

Resource	URL
Live app	https://bondnest.up.railway.app/
GitHub	https://github.com/rproject1324/BondNest

No portfolio or demo-video URLs were supplied; they are omitted rather than invented.

19. Acknowledgements

BondNest is documented here as a single-author, second-year project. The implementation, deployment, and this technical record are the work of Tolentino, Lawrence Dave P.